
Chapter 2

Introduction to Relational Model



Outline

- ▶ Structure of Rational Database
- ▶ Database Schema
- ▶ Keys
- ▶ Schema Diagrams
- ▶ Relational Query Language
- ▶ Relational Operations



Example of a Relation

attributes
(or columns)

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

tuples
(or rows)

Figure 2.1 The *instructor* relation

Each row of this table records information about an instructor, consisting of the instructor's *ID*, *name*, *dept_name* and *salary*.



Example of a Relation(Cont.)

- ▶ Similarly, the **course** table stores information about *courses*, consisting of a *course id*, *title*, *dept_name* and *credits*, for each course.
 - ▶ Each course is identified by the value of the column *course_id*.
- ▶ The table **prereq** stores the prerequisite courses for each course.
 - ▶ Each row consists of a pair of course identifiers such that the second course is a prerequisite for the first course.

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Figure 2.2 The *course* relation.

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Figure 2.3 The *prereq* relation.



Structure of Relational Databases

- ▶ A **relational database** consists of a collection of *tables*.
- ▶ In general, *a row in a table* represents a relationship among a set of values.
- ▶ There is a close correspondence between *the concept of table* and *the mathematical concept of relation*, from which the relational data model takes its name.
- ▶ Thus, in the relational model the term **relation** is used to *refer to a table*, while the term **tuple** is used to *refer to a row*. Similarly, the term **attribute** *refers to a column of a table*.
- ▶ The **order** in which tuples appear in a relation is *irrelevant*, since a relation is *a set of tuples*.



Relation Instance

- ▶ We use the term **relation instance** to refer to *a specific instance of a relation*, i.e., containing a specific set of rows.
- ▶ The **instance of instructor** shown in Figure 2.1 has 12 tuples, corresponding to 12 instructors.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation



Relations are Unordered

- ▶ The order in which tuples appear in a relation is *irrelevant*, since a relation is a set of tuples.
- ▶ Whether the tuples of a relation are listed in sorted order, as in Figure 2.1, or are unsorted, as in Figure 2.4, does not matter

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Figure 2.4 Unsorted display of the *instructor* relation.

Domain

- ▶ For each attribute of a relation, there is *a set of permitted values*, called the **domain** of that attribute.
 - ▶ The domain of the *salary* attribute of the instructor relation is the set of all possible salary values.
 - ▶ The domain of the *name* attribute is the set of all possible instructor names.
- ▶ We require that, for all relations r , the domains of all attributes of r be **atomic**. A domain is atomic if *elements of the domain are considered to be indivisible units*.
 - ▶ Suppose the table *instructor* had an attribute *phone number*. Then the domain of *phone number* would not be atomic, since an element of the domain is a set of phone numbers.



Null Value

- ▶ The **null value** is a special value that signifies that *the value is unknown or does not exist*.
- ▶ Suppose we include the attribute *phone number* in the *instructor* relation.
- ▶ It may be that an instructor does not have a *phone number* at all, or that the telephone number is unlisted.
- ▶ The **special value null** is a member of every domain
- ▶ The null value causes *complications in the definition of many operations*



Relation Schema

- ▶ **Database schema:** the logical design of the database
- ▶ **Database instance:** a snapshot of the data in the database at a given instant in time
- ▶ The concept of a *relation* corresponds to the programming-language notion of a *variable*.
- ▶ The concept of a *relation schema* corresponds to the programming-language notion of *type definition*.
- ▶ The concept of a *relation instance* corresponds to the programming-language notion of *a value of a variable*.



Relation Schema

- ▶ $A_1, A_2, \dots, A_n$ are *attributes*
- ▶ $R = (A_1, A_2, \dots, A_n)$ is a *relation schema*

Example:

$instructor = (ID, name, dept_name, salary)$

- ▶ Formally, given sets $D_1, D_2, \dots, D_n$ a **relation** r is a subset of

$$D_1 \times D_2 \times \dots \times D_n$$

Thus, a relation is a set of n -tuples $(a_1, a_2, \dots, a_n)$ where each $a_i \in D_i$



Relation Schema (cont.)

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Figure 2.5 The *department* relation.

- ▶ *department* (*dept_name*, *building*, *budget*)
- ▶ Note that the attribute *dept_name* appears in both the *instructor* schema and the *department* schema.
- ▶ Using **common attributes** in relation schemas is one way of *relating tuples of distinct relations*.



University Database Example

- ▶ Let us continue with our university database example.
- ▶ We need a relation to describe *each individual offering, or section, of the class*. The schema is

section (course_id , sec_id , semester, year, building, room_number, time_slot_id)

- ▶ We need a relation to describe the *association between instructors and the class sections that they teach*.

teaches (ID, course_id , sec_id , semester , year)



University Database Example

<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>room_number</i>	<i>time_slot_id</i>
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Figure 2.6 The *section* relation.

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Figure 2.7 The *teaches* relation.

- ▶ *section* (*course_id* , *sec_id* , *semester* , *year* , *building*, *room_number*, *time_slot_id*)
- ▶ *teaches* (*ID*, *course_id* , *sec_id* , *semester* , *year*)



University Database Example.

- ▶ There are many more relations maintained in a real university database.

- *student* (*ID, name, dept_name, tot_cred*)
- *advisor* (*s_id, i_id*)
- *takes* (*ID, course_id, sec_id, semester, year, grade*)
- *classroom* (*building, room_number, capacity*)
- *time_slot* (*time_slot_id, day, start_time, end_time*)



Database

- ▶ A database consists of **multiple relations**
- ▶ Information about an enterprise is broken up into parts:
instructor *student* *advisor*
- ▶ Bad design:
univ (instructor -ID, name, dept_name, salary, student_Id, ...)
results in
 - ▶ ***repetition of information*** (e.g., two students have the same instructor)
 - ▶ ***need for null values*** (e.g., represent an student with no advisor)
- ▶ **Normalization theory** (Chapter 7) deals with ***how to design “good” relational schemas***



Keys

- ▶ We must have *a way to specify how tuples within a given relation are distinguished.*
- ▶ The values of the attribute values of a tuple must be such that they can *uniquely identify* the tuple.
- ▶ A **superkey** is a set of one or more attributes that, taken collectively, allow us to identify uniquely a tuple in the relation.
- ▶ Formally, K is a **superkey** for r if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$.
 - ▶ Example: $\{ID\}$ and $\{ID, name\}$ are both superkeys of *instructor*.
 - ▶ $\{name\}$ attribute is not a superkey of *instructor*



Candidate Keys

- ▶ A **superkey** may contain extraneous attributes.
- ▶ If K is a superkey, then so is any superset of K .
- ▶ The superkeys, for which no proper subset is a superkey, are called **candidate keys**.
 - ▶ Example: $\{ID\}$ is a candidate key for *Instructor*
- ▶ It is possible that *several distinct sets of attributes could serve as a candidate key*.
 - ▶ Suppose that a combination of *name* and *dept_name* is sufficient to distinguish among members of the *instructor* relation. Then, both $\{ID\}$ and $\{name, dept_name\}$ are candidate keys.



Primary Key

- ▶ We shall use the term **primary key** to denote **a candidate key** that is *chosen by the database designer as the principal means of identifying tuples within a relation.*
- ▶ Primary keys must be chosen with care.
 - ▶ Example: In the United States, the social-security number attribute of a person would be a candidate key.
 - ▶ Example: International enterprises must generate their own unique identifiers since non-U.S. residents usually do not have social-security numbers.
 - ▶ An alternative is to use *some unique combination of other attributes as a key.*



Primary Key

- ▶ The **primary key** should be chosen such that its attribute values are *never, or very rarely, changed*.
 - ▶ The *address* field of a person should not be part of the primary key, since it is likely to change.
 - ▶ *Social-security numbers*, on the other hand, are guaranteed never to change.
 - ▶ *Unique identifiers* generated by enterprises generally do not change.
- ▶ It is customary to list the primary key attributes of a relation schema before the other attributes; primary key attributes are also *underlined*.
 - ▶ *department (dept_name, building, budget)*



Foreign key

- ▶ A relation, say r_1 , may include among its attributes the primary key of another relation, say r_2 . This attribute is called a **foreign key** from r_1 referencing r_2 .
- ▶ The relation r_1 is also called the **referencing relation** of the foreign key dependency, and r_2 is called the **referenced relation** of the foreign key.

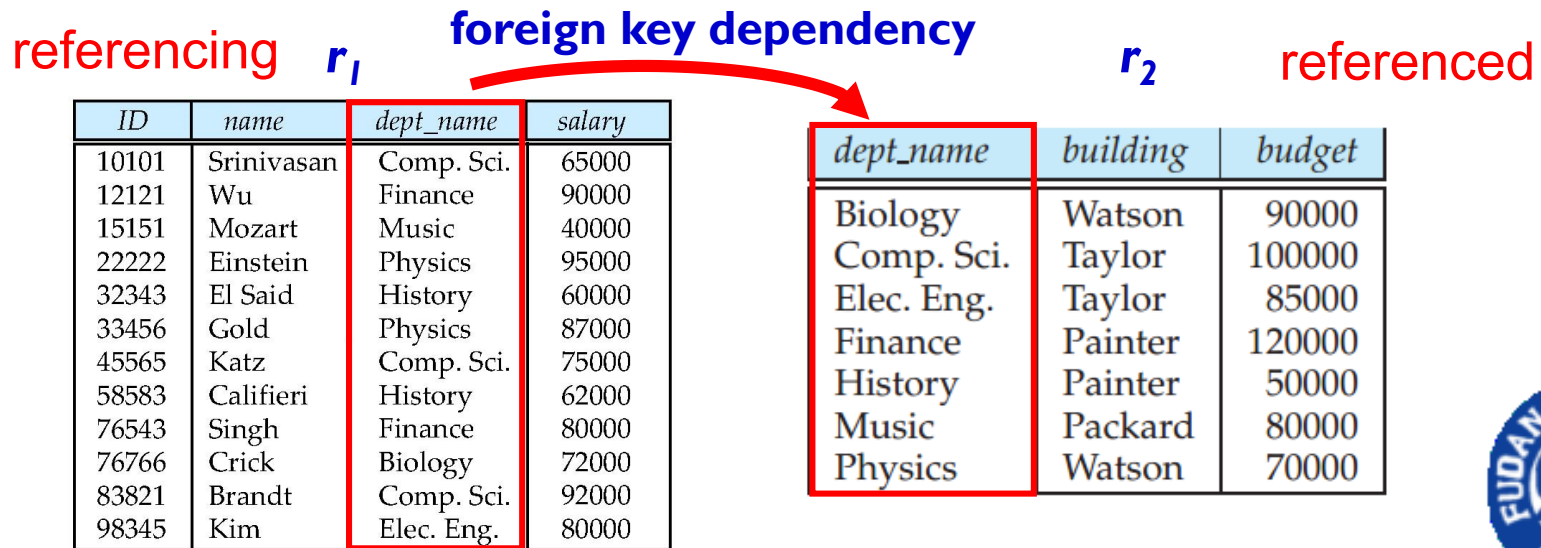


Figure 2.5 The department relation.

Referential Integrity Constraint

- ▶ A **referential integrity constraint** requires that the values appearing in specified attributes of any tuple in the referencing relation **also appear** in specified attributes of at least one tuple in the referenced relation.

section (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*,
time_slot_id)

teaches (*ID*, *course_id*, *sec_id*, *semester*, *year*)

- ▶ If a particular (*course_id*, *sec_id*, *semester*, *year*) combination appears in *section*, then the same combination must appear in *teaches* .

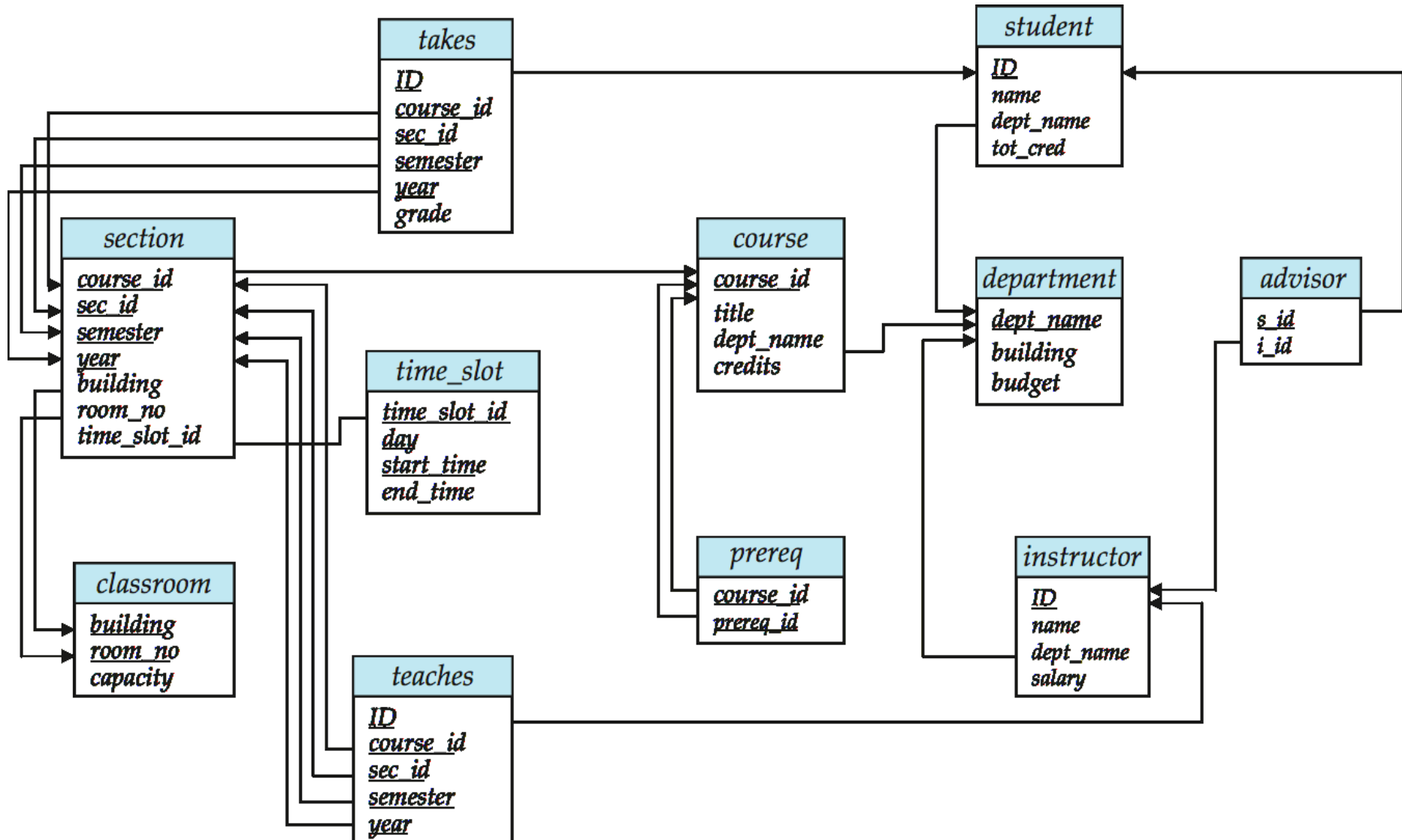


Schema Diagrams

- ▶ A *database schema*, along with *primary key* and *foreign key dependencies*, can be depicted by **schema diagrams**.
 - ▶ Each *relation* appears as a box, with the relation name at the top in blue, and the attributes listed inside the box.
 - ▶ *Primary key* attributes are shown underlined.
 - ▶ *Foreign key dependencies* appear as arrows from the foreign key attributes of the referencing relation to the primary key of the referenced relation.
- ▶ Many database systems provide **design tools** with a graphical user interface for creating schema diagrams.



Schema Diagram for University Database



Schema Diagram for University Database

- ▶ The enterprise that we use in the examples in later chapters is a university.

classroom(building, room_number, capacity)
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
instructor(ID, name, dept_name, salary)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
teaches(ID, course_id, sec_id, semester, year)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)
advisor(s_ID, i_ID)
time_slot(time_slot_id, day, start_time, end_time)
prereq(course_id, prereq_id)

Figure 2.9 Schema of the university database.



Relational Query Languages

- ▶ A **query language** is a language in which a user *requests information from the database*.
- ▶ Procedural vs. non-procedural, or declarative
 - ▶ In a **procedural language**, the *user instructs the system to perform a sequence of operations on the database to compute the desired result*.
 - ▶ In a **nonprocedural language**, the *user describes the desired information without giving a specific procedure for obtaining that information*.
- ▶ The **relational algebra** is procedural, whereas the **tuple relational calculus** and **domain relational calculus** are nonprocedural.



Relational Operations

- ▶ All procedural relational query languages provide *a set of operations* that can be applied to either a single relation or a pair of relations.
- ▶ The most frequent operation is the **selection** of specific tuples from *a single relation* (say *instructor*) that satisfies some *particular predicate* (say $salary > \$85,000$).

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
12121	Wu	Finance	90000
22222	Einstein	Physics	95000
33456	Gold	Physics	87000
83821	Brandt	Comp. Sci.	92000

Figure 2.10 Result of query selecting *instructor* tuples with salary greater than \$85000.



Selection of tuples

- Relation r :

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

- Select tuples with
 $A=B$ and $D > 5$

- $\sigma_{A=B \text{ and } D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10



Selection of Columns (Attributes)

- ▶ Another frequent operation is to **select certain attributes** (columns) from a relation. The result is a new relation having only those selected attributes.

<i>ID</i>	<i>salary</i>
10101	65000
12121	90000
15151	40000
22222	95000
32343	60000
33456	87000
45565	75000
58583	62000
76543	80000
76766	72000
83821	92000
98345	80000

Figure 2.11 Result of query selecting attributes *ID* and *salary* from the *instructor* relation.



Selection of Columns (Attributes)

- Relation r :

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

- Select A and C
 - Projection
 - $\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2



Joining two relations – Cartesian Product

- Relations r, s :

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

- $r \times s$:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b



Union of two relations

- Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- $r \cup s$:

A	B
α	1
α	2
β	1
β	3



Set difference of two relations

- Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- $r - s$:

A	B
α	1
β	1



Intersection of two relations

- Relation r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- $r \cap s$

A	B
α	2



Joining two relations – Natural Join

- ▶ Let r and s be relations on schemas R and S respectively.
- ▶ Then, the “**natural join**” of relations R and S is **a relation on schema $R \cup S$** obtained as follows:
 - ▶ Consider *each pair* of tuples t_r from r and t_s from s .
 - ▶ If t_r and t_s have the *same value on each of the attributes in $R \cap S$* , add a tuple t to the result, where
 - ▶ t has the same value as t_r on r
 - ▶ t has the same value as t_s on s



Joining two relations – Natural Join

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

The *instructor* relation

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Figure 2.5 The *department* relation.

Natural Join



<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
12121	Wu	90000	Finance	Painter	120000
15151	Mozart	40000	Music	Packard	80000
22222	Einstein	95000	Physics	Watson	70000
32343	El Said	60000	History	Painter	50000
33456	Gold	87000	Physics	Watson	70000
45565	Katz	75000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
76543	Singh	80000	Finance	Painter	120000
76766	Crick	72000	Biology	Watson	90000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000

Figure 2.12 Result of natural join of the *instructor* and *department* relations.



Natural Join Example

- Relations r, s :

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

- Natural Join
- $r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ



Relational Algebra

- ▶ The relational algebra defines a set of operations on relations, paralleling the usual algebraic operations such as **addition, subtraction** or **multiplication**, which operate on numbers.

Symbol (Name)	Example of Use
σ (Selection)	$\sigma_{\text{salary} \geq 85000}(\text{instructor})$
	Return rows of the input relation that satisfy the predicate.
Π (Projection)	$\Pi_{ID, salary}(\text{instructor})$
	Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
$\bowtie$ (Natural Join)	$\text{instructor} \bowtie \text{department}$
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.
$\times$ (Cartesian Product)	$\text{instructor} \times \text{department}$
	Output all pairs of rows from the two input relations (regardless of whether or not they have the same values on common attributes)
$\cup$ (Union)	$\Pi_{name}(\text{instructor}) \cup \Pi_{name}(\text{student})$
	Output the union of tuples from the two input relations.

Duplicate tuples

- ▶ Sometimes, the result of a query contains **duplicate tuples**.
- ▶ Certain relational languages adhere strictly to the mathematical definition of a set and *remove duplicates*.
- ▶ Others, in consideration of the *relatively large amount of processing* required to remove duplicates from large result relations, *retain duplicates*.



Homework

► 2.6, 2.9, 作业请于下次上课时交给助教。

2.6 Consider the following expressions, which use the result of a relational algebra operation as the input to another operation. For each expression, explain in words what the expression does.

- a. $\sigma_{year \geq 2009}(takes) \bowtie student$
- b. $\sigma_{year \geq 2009}(takes \bowtie student)$
- c. $\Pi_{ID, name, course_id}(student \bowtie takes)$

2.9 Consider the bank database of Figure 2.15.

- a. What are the appropriate primary keys?
- b. Given your choice of primary keys, identify appropriate foreign keys.

branch(branch_name, branch_city, assets)
customer(customer_name, customer_street, customer_city)
loan(loan_number, branch_name, amount)
borrower(customer_name, loan_number)
account(account_number, branch_name, balance)
depositor(customer_name, account_number)

Figure 2.15 Banking database for Exercises 2.8, 2.9, and 2.13.



The End

